

AGENCYOS

PHASE 4 - ORCHESTRATOR / ROUTER AGENT

Task 2: Full UI Design + UI Prototype

ROUTING + EXECUTION CONTROL + MULTI-MODEL POLICY + TOOL GATING + RETRY/FALLBACK + IMPLEMENTATION + TESTING SPECIFICATION

Official Role	Orchestrator / Router Agent
Phase	Phase 4 - Task 2
Primary Mission	Route and execute Phase 4 AI work through the correct specialist, eligible model/provider and authorized tool path while preserving state, versions, permissions, retries, evidence and audit
Specialists in Scope	PM Agent, UI Designer Agent, QA / Prototype Validation Agent, Prototype Agent, Finance Agent when M2 activates
Core Boundary	Orchestrator controls AI execution routing; it does not become the specialist, Coordination workflow owner, Policy Engine, Admin or Client
Model Principle	Capability-driven, model-agnostic routing; provider/model selection is configuration/policy, not hard-coded business logic
Hard Rule	No routing decision can bypass business policy, permissions, Admin/client gates, payment verification or exact-version requirements
Authoritative Outcome	Structured validated result + routing/execution trace; generated text alone never changes authoritative business state

This document applies the existing AgencyOS Orchestrator / Router contract to the locked Phase 4 workflow. It preserves the distinction between Orchestrator execution control and Coordination workflow handoff/dependency control.

1. Mission

The Phase 4 Orchestrator / Router receives structured tasks/events from AgencyOS workflow coordination, understands the required capability, selects the eligible specialist, selects an eligible model/provider, verifies permissions/tool access/context, builds the execution envelope, dispatches the work, validates the structured result, performs bounded retry/fallback when permitted, and returns a traceable outcome.

It is the AI workforce execution-control layer, not the owner of UI design, PM communication, QA decisions, prototype construction, Finance authority or workflow approvals.

2. Orchestrator vs Coordination / Workflow Layer

Question	Orchestrator / Router	Coordination / Workflow
Which specialist executes?	Primary selection/eligibility	Requests/records workflow need
Which model/provider?	Primary	No
Which tool route?	Permission/tool execution gate	Tracks dependency/outcome
Workflow next step?	Returns execution result	Primary
Cross-agent handoff/dependency?	Execution-level support	Primary workflow layer
Retry/fallback route?	Primary for AI/provider execution	Tracks business/job outcome
Business/domain work?	No	No
Admin/Client approval?	No	No - humans/approval system
Audit focus	Routing + execution	Workflow + handoff

Phase 4 master flow sometimes groups 'Orchestrator / Coordination' as backend infrastructure. Implementation must still keep responsibilities separable so routing logic does not accidentally become business workflow authority.

3. Phase 4 Specialist Routing Map

Task / event	Specialist route	Orchestrator responsibility
Task 2 client communication / clarification	PM Agent	Select PM capability/model; enforce communication permission/schema
Complete high-fidelity UI	UI Designer Agent	Route only when valid Designer activation condition exists
Design QA	QA capability / QA Agent	Route exact UI version; enforce independent QA result schema
Prototype build	Prototype Agent	Route only from exact client-approved locked UI version
Prototype validation	QA / Prototype Validation Agent	Route exact build; enforce independent validation
M2 finance workflow after Phase4Completed	Finance Agent	Route Finance task without granting verification authority
Admin/client decisions	Human / approval workflow	Never route as autonomous AI approval
Phase transitions / handoffs	Coordination / workflow	Orchestrator executes requested specialist work; Coordination owns next-step workflow

4. Task Intake Contract

- ☐ Task ID and correlation ID.
- ☐ organization / tenant context.
- ☐ project_id and current Phase 4 task/state.
- ☐ source actor/agent/system.
- ☐ task type and intent.
- ☐ required specialist capability.
- ☐ required output schema.
- ☐ exact source artifact/version references.
- ☐ priority/deadline where applicable.
- ☐ authorized context references.
- ☐ policy/permission constraints.
- ☐ data classification.
- ☐ required tools/integrations.
- ☐ idempotency key and retry budget.
- ☐ acceptance criteria

Malformed or incomplete task envelopes must be rejected before model/tool execution. The Orchestrator must not repair missing business facts by guessing.

5. Phase 4 Intent & Capability Understanding

- ☐ Distinguish communication, design, QA, prototype build, prototype validation and finance work.
- ☐ Detect whether task is informational, generative, analytical, tool-using or side-effecting.
- ☐ Detect exact version/build requirements.
- ☐ Detect human approval prerequisites.
- ☐ Detect whether Designer conditional activation is valid.
- ☐ Detect whether Finance task is allowed before/after Phase4Completed.
- ☐ Detect whether a task attempts to bypass QA/Admin/client/payment gates.
- ☐ Detect routing-changing ambiguity and escalate rather than guessing.

6. Capability Registry Requirements

Registry item	Must know
Agent identity/version	registered ID, enabled/disabled, current version
Task support	supported Phase 4 task types
Input/output schemas	strict structured contract
Tools	required/optional tools and dispatchability
Permissions	project/tenant/action permissions
Data classes	what context/data the agent may access
Autonomy	allowed work classes / approval restrictions
Health	available/degraded/disabled/unhealthy
Constraints	platform, Figma/build, QA, Finance or other restrictions

7. Multi-Model / Provider Routing

AgencyOS remains model-agnostic. The Orchestrator chooses from currently configured eligible providers/models according to capability, policy, health, quality, latency, privacy and cost.

Phase 4 workload	Typical capability profile	Routing priority
PM communication / clarification	GENERAL_HIGH_QUALITY + STRUCTURED_OUTPUT	tone, reliability, policy adherence, cost
UI Designer	MULTIMODAL + LONG_CONTEXT + STRUCTURED_OUTPUT	UI understanding/design reasoning, quality, context
Design QA	REASONING_HIGH + MULTIMODAL + STRUCTURED_OUTPUT	independent analysis, evidence, schema correctness
Prototype Agent	CODING_HIGH + TOOL_AGENT + STRUCTURED_OUTPUT	tool use, coding/build reliability, cost
Prototype Validation	REASONING_HIGH + TOOL_AGENT + MULTIMODAL	test analysis, build/evidence validation
Finance Agent	STRUCTURED_OUTPUT + GENERAL_HIGH_QUALITY	correctness, determinism, policy compliance
Orchestrator self-routing	GENERAL_HIGH_QUALITY / REASONING_HIGH	routing correctness, structured decisions

Provider/model IDs are configuration. Business code should ask for capability tiers, not permanent model names.

8. Routing Decision Pipeline

TASK → CAPABILITY REQUIREMENTS → AGENT ELIGIBILITY → POLICY/PERMISSION FILTER → TOOL FILTER
→ MODEL/PROVIDER CANDIDATES → HEALTH / QUOTA → COST / QUALITY / LATENCY → PRIMARY ROUTE +
FALLBACK → EXECUTION ENVELOPE → EXECUTE → VALIDATE → RETRY / FALLBACK / ESCALATE

9. Selection Algorithm - Phase 4 Micro Steps

- ☐ Validate task schema and correlation.
- ☐ Validate tenant/project scope.
- ☐ Resolve capability and specialist class.
- ☐ Load candidate agents.
- ☐ Filter disabled/unavailable agents.
- ☐ Filter by task support.
- ☐ Filter by permissions/data classification.
- ☐ Filter by required tool availability.
- ☐ Filter by output schema compatibility.
- ☐ Enforce Phase 4 state/version gates.
- ☐ Load provider/model candidates.
- ☐ Filter disabled/unhealthy/unauthorized providers.
- ☐ Check context window, tool support and structured output capability.
- ☐ Apply configured quality/cost/latency/privacy policy.
- ☐ Select primary route and permitted fallback.
- ☐ Persist routing decision before dispatch.

10. Phase 4 Routing Guards

Guard	Required behavior
Phase 3 incomplete	Block Task 2 specialist execution that requires valid Phase 4 start
UI Designer task without valid activation reason	Do not route Designer
Design QA on unknown/stale UI version	Reject or reconcile exact version first
Admin UI approval missing	Block PM client-share execution
Client UI approval missing	Block Prototype Agent build task
Prototype build source differs from locked UI	Reject
Prototype QA on wrong build	Reject/reconcile
Admin prototype approval missing	Block PM client prototype share
Phase4Completed missing	Do not route M2 Finance activation
M2 PaymentVerified missing	Block Task 3 / Phase 5 start route

11. Execution Envelope - Required Fields

- ☐ task_id / correlation_id.
- ☐ organization/project scope.
- ☐ source actor and destination specialist.
- ☐ task type and validated intent.
- ☐ exact requirement/scope/UI/build version references.
- ☐ authorized context refs only.
- ☐ required output schema.
- ☐ acceptance criteria.
- ☐ tool permissions.
- ☐ policy version / approval references.
- ☐ priority/deadline.
- ☐ retry budget and timeout.
- ☐ idempotency key.
- ☐ data-classification/security metadata.
- ☐ selected model/provider and selection reason.

The execution envelope must carry exact UI/build versions for Phase 4. 'Latest' is not acceptable where an approval is tied to a specific artifact.

12. Context & Memory Rules

- ☐ Retrieve only relevant authorized project context.
- ☐ Prefer canonical structured records over chat summaries.
- ☐ Reuse locked Phase 3 baseline rather than re-asking/re-generating it.
- ☐ Include exact current UI/build/revision identifiers.
- ☐ Pass only minimum necessary context to provider/model.
- ☐ Preserve provenance for client/Admin/QA decisions.
- ☐ Include prior failure context on retry.
- ☐ Reject stale/conflicting context where it can change outcome.
- ☐ Never leak another client's/project's memory.

13. Tool & Permission Gate

- ☐ Check agent identity and project permission.
- ☐ Check task/action permission.
- ☐ Check tool is registered and dispatchable for this autonomy/work class.
- ☐ Check integration permission and environment.
- ☐ Check data classification.
- ☐ Validate tool arguments.
- ☐ Treat tool/external responses as untrusted until validated.
- ☐ Block hidden/non-dispatchable write tools.
- ☐ Do not let model output grant itself permissions.
- ☐ Audit important permission allow/deny decisions

Current AgencyOS direction favors READ capability first and governed dispatch of write/action tools. A registered tool is not automatically dispatchable.

14. Execution Control

- ☐ Persist execution start.
- ☐ Apply timeout.
- ☐ Track attempts/status.
- ☐ Track important tool calls.
- ☐ Validate provider response correlation.
- ☐ Validate structured output schema.
- ☐ Validate required artifact/version references.
- ☐ Validate completion criteria.
- ☐ Reject incomplete/unsupported result.
- ☐ Return validated result plus execution metadata.
- ☐ Do not directly mutate business authority state from free-form generated text.

15. Output Validation by Specialist Type

Specialist	Orchestrator validates
PM	message/result schema, exact artifact version, communication permission; not truth of human approval unless authoritative event exists
UI Designer	UI version metadata, scope/version refs, activation reason, artifact refs; not self-QA
QA	exact UI/build under test, result/evidence/defect schema, independent status
Prototype	build ID, source UI version, artifact ref, build status/limitations; not QA PASS
Finance	invoice/milestone/payment match schema; no autonomous Admin verification claim

16. Retry Policy

- ☐ Classify failure before retry.
- ☐ Determine whether retry is safe.
- ☐ Check retry budget.
- ☐ Use configured backoff.
- ☐ Preserve task/correlation IDs.
- ☐ Increment attempt counter.
- ☐ Carry prior failure context.
- ☐ Prevent duplicate non-idempotent side effects.
- ☐ Do not change provider/route unless policy allows.
- ☐ Stop and escalate after exhaustion.

17. Fallback Policy

- ☐ Fallback must provide equivalent required capability.
- ☐ Fallback specialist must still be the correct domain owner; provider fallback must not change business authority.
- ☐ Permissions/data classification must remain valid.
- ☐ Required output/tool/schema support must remain available.
- ☐ Cost/quality thresholds must remain within policy.
- ☐ Record primary failure and fallback reason.
- ☐ Preserve correlation and exact artifact/version refs.
- ☐ Escalate if no safe fallback exists.

18. Side-Effect Safety

- ☐ Detect tool calls that create/update external or authoritative state.
- ☐ Require idempotency key.
- ☐ Confirm authorization immediately before dispatch.
- ☐ Persist external request/reference.
- ☐ Validate external response.
- ☐ Do not blindly retry uncertain side effects.
- ☐ Use durable outbox/job/event pattern where required.
- ☐ Reconcile uncertain execution before retry.
- ☐ Escalate ambiguous financial/destructive outcomes

Examples: do not duplicate client messages, Figma writes, prototype build jobs, invoice creation, payment credit or approval actions on retry/replay.

19. Phase 4 Event / Job Routing Map

Event / task	Route / behavior
Phase3Completed / Task2Started	Coordination initializes workflow; Orchestrator routes PM start communication and eligible Phase 4 work
UIDesignReady	Route exact UI version to Design QA
UIDesignQACHangesRequired	Route targeted correction to UI Designer
UIDesignQAPassed	Return validated QA result; workflow creates Admin review
AdminUIEditRequested	Route exact edit package to Designer; then QA again
AdminUIApproved	Route PM client-share task only if workflow gate valid
ClientUIChangeApproved	Route governed Designer revision
ClientUIApproved	Route PrototypeBuildRequested from exact locked UI
PrototypeBuildReady	Route exact build to Prototype Validation QA
PrototypeQACHangesRequired	Route defects to Prototype Agent
PrototypeQAPassed	Workflow creates Admin prototype review
AdminPrototypeEditRequested	Route revised build work to Prototype Agent; QA again
AdminPrototypeApproved	Route PM client-share task
ClientPrototypeChangeApproved	Route controlled prototype revision
ClientPrototypeApproved	Workflow evaluates Phase4 completion
Phase4Completed	Route Finance M2 activation
M2PaymentVerified	Route PM Task 3 communication / Phase 5 eligible execution

20. Conditional UI Designer Routing

- ☐ Route Designer on initial full-UI job only after valid Phase 4 baseline.
- ☐ Route Designer on confirmed coverage gap.
- ☐ Route Designer on Design QA defect.
- ☐ Route Designer on Admin EDIT.
- ☐ Route Designer on PM-classified allowed client UI revision.
- ☐ Do not route Designer for ambiguous business rule - PM clarification first.
- ☐ Do not route Designer for new scope until Change Request approval.
- ☐ Do not route Designer for prototype-only code defect.
- ☐ After client UI lock, Designer remains inactive unless true source-UI design change is governed.

21. QA Independence Guard

- ☐ Designer cannot be selected as validator of its own UI.
- ☐ Prototype Agent cannot be selected as validator of its own build.
- ☐ QA result must come from QA-capable identity with proper permissions.
- ☐ Old QA PASS cannot automatically validate a changed version/build.
- ☐ Routing preserves defect → fix → QA retest separation.

22. Finance & Payment Gate Routing

- ☐ Finance task activates only after valid Phase4Completed.
- ☐ Orchestrator may route Finance calculation/invoice/matching work.
- ☐ Orchestrator must reject any AI route claiming final payable-payment verification when locked policy requires Admin.
- ☐ Payment proof/client message cannot become authoritative verification.
- ☐ M2PaymentVerified event must come from authorized Admin-verification workflow.
- ☐ Only after verified event may PM Task 3 / Phase 5 execution route

Model confidence cannot override financial authority. The Orchestrator validates authority provenance, not just text content.

23. Human / Admin Escalation Conditions

- ☐ No eligible specialist.
- ☐ No eligible provider/model.
- ☐ Policy/permission conflict.
- ☐ Repeated execution failure.
- ☐ Critical timeout.
- ☐ Ambiguous high-risk routing.
- ☐ Conflicting authoritative records.
- ☐ Stale/unresolvable version conflict.
- ☐ Unsafe side-effect uncertainty.
- ☐ Unclear scope vs revision classification.
- ☐ Financial verification ambiguity

Escalation package should include task, correlation, exact context/artifact refs, attempted routes, failures, policy/permission reason and next-action recommendation.

24. Failure Matrix

Failure	Required behavior
Malformed task	Reject before dispatch
No capable agent	Escalate; never guess
Agent unavailable	Permitted fallback or escalation
Provider/model failure	Retry/fallback according to policy
Timeout	Safe retry; otherwise reconcile/escalate
Permission denied	Stop and return structured authorization error
Required tool non-dispatchable	Block/alternate approved route/manual task
Invalid output	Reject; retry/fallback if safe
Artifact/version missing	Treat result incomplete
Conflicting authoritative state	Escalate to owner/workflow
Uncertain side effect	Do not blindly retry
Policy conflict	Stop; route to Policy/Admin

25. Security & Data Governance

- ☐ Tenant / organization isolation.
- ☐ Project-scoped context and tool access.
- ☐ Least privilege.
- ☐ Data-classification-aware routing.
- ☐ Authorized memory only.
- ☐ Secret isolation from prompts/logs/client outputs.
- ☐ Tool arguments and external outputs validated.
- ☐ Provider output cannot grant permission.
- ☐ Cross-client/project context leakage blocked.
- ☐ Agent-to-agent calls authenticated/authorized.
- ☐ Critical routing and permission decisions audited.

26. Cost, Latency & Quality Control

- ☐ Admin-configurable global/provider/agent/project/task budgets.
- ☐ Use cheaper eligible tier for routine deterministic/simple tasks.
- ☐ Use stronger capability for high-impact design/QA/complex reasoning tasks.
- ☐ Do not route expensive AI for deterministic state transitions/calculations.
- ☐ Reuse stable context and approved artifacts.
- ☐ Avoid sending entire project history when minimum context suffices.
- ☐ Track provider/model usage/tokens/cost where available.
- ☐ Alert/escalate or route lower-cost approved tier when budget policy requires.
- ☐ Never choose the cheapest route when policy requires higher quality.

27. Observability & Audit

- ☐ task_id / correlation_id.
- ☐ source/destination specialist.
- ☐ candidate and selected route.
- ☐ model/provider selected where applicable.
- ☐ routing reason and policy/config version.
- ☐ exact UI/build/scope refs.
- ☐ execution timestamps and duration.
- ☐ attempt count.
- ☐ tool authorization decisions where required.
- ☐ failure/retry/fallback history.
- ☐ result validation outcome.
- ☐ escalation record.
- ☐ usage/cost metrics where available.
- ☐ no raw secrets in logs.

28. Admin Panel - Orchestrator / Router Visibility

Admin surface	Must show / control
Routing Dashboard	task volume, success/failure, current routes, provider health, queue/backlog
Task Trace	correlation chain, source, specialist, model/provider, attempts, exact artifact refs
Agent Registry	enabled status, capabilities, schemas, tools, permissions, health
Model / Provider Routing	capability profile, primary/fallback, cost/quality/latency policy
Tool Permissions	registered vs dispatchable tools, work-class restrictions
Phase 4 Trace	PM/Designer/QA/Prototype/Finance routed executions with current task gate
Failure Queue	timeouts, invalid output, no-route, permission, tool, provider failures
Retry / Fallback	attempt history, backoff, fallback route, result
Budget / Usage	agent/provider/project cost and thresholds where data exists
Escalations	reason, attempted routes, next action and human owner
Audit	routing-policy changes, agent/tool/provider configuration changes

29. Data Contracts

Entity	Minimum fields
AgentCapability	agent_id, version, task_types, input/output schemas, tools, permissions, data classes, availability, constraints
RoutingDecision	decision_id, task/correlation, candidate agents/models, selected agent/model/provider, policy version, reason, constraints, timestamp
ExecutionEnvelope	task/correlation, tenant/project, source/destination, task type, intent, context/version refs, schema, criteria, tool permissions, policy, retry/idempotency
ExecutionRecord	execution_id, route, provider/model, attempt, start/end, status, tool calls, result ref, error, fallback_from/to, validation result
RouteFailure	task/execution, failure class, provider/agent/tool, retryable, evidence, action
FallbackRecord	primary route, fallback route, reason, policy, result
OrchestratorEscalation	task, context refs, attempted routes, failures, policy refs, owner, recommendation
UsageCostRecord	execution, provider/model, usage units/tokens, reported cost/estimate, currency/source

30. Technical Implementation Plan

OR4-I01 Existing Orchestrator Discovery

Inspect current Orchestrator, Coordination layer, agent registry, provider registry, tool registry, policies, job/event system, execution traces and Phase 4 integrations before creating anything new.

OR4-I02 Phase 4 Task Taxonomy

Define explicit Phase 4 task types for PM, Designer, Design QA, Prototype, Prototype QA and Finance.

OR4-I03 Capability Registry

Register/verify specialist capability, schemas, tools, permissions, autonomy and health metadata.

OR4-I04 Routing Policy Layer

Implement capability-driven provider/model eligibility, scoring/config, preferred/fallback routes and policy versioning.

OR4-I05 Phase 4 State Guards

Enforce exact phase/UI/build/Admin/client/payment gate prerequisites before dispatch.

OR4-I06 Execution Envelope

Implement validated structured envelope with exact version/context refs, acceptance criteria, tools, policy and idempotency.

OR4-I07 Context Resolver

Load minimum authorized canonical context with provenance and stale-version detection.

OR4-I08 Tool Permission Gate

Enforce registered/dispatchable tool metadata, permissions, data classification and work-class restrictions.

OR4-I09 Dispatch / Execution Control

Persist decision, dispatch specialist/model, timeout, status/tool trace and result ingestion.

OR4-I10 Output Validator

Enforce schemas, version/artifact refs, authority provenance and acceptance criteria.

OR4-I11 Retry / Backoff

Failure classification, safe retry, bounded attempts and correlation preservation.

OR4-I12 Fallback

Equivalent-capability fallback with permission/policy/cost/quality checks and trace.

OR4-I13 Side-Effect Safety

Idempotency/outbox/reconciliation for messages, Figma writes, builds, Finance records and other non-idempotent actions.

OR4-I14 Conditional Designer Routing

Implement valid activation/no-activation branches for Designer.

OR4-I15 QA Independence

Prevent creator-as-validator and force re-QA of changed versions/builds.

OR4-I16 Finance Authority Guard

Route Finance work while enforcing Admin-only verification and M2 gate provenance.

OR4-I17 Admin Observability

Expose routing/execution/failure/fallback/cost/Phase 4 traces and configuration.

OR4-I18 Security Hardening

Tenant/project isolation, secrets, agent/tool permissions and audit.

OR4-I19 Automated Test Harness

Unit/integration/event/tool/security/routing/fallback/idempotency tests.

OR4-I20 Phase 4 E2E

Run exact Task 2 workflow through all routed specialists, failures/retries and M2 gate.

31. Test Matrix

Area	Required coverage
Task intake	valid/malformed envelope, missing project/version, wrong phase
Agent routing	PM/Designer/QA/Prototype/Finance task to correct specialist
Conditional Designer	valid activation and no-activation branches
Model/provider	policy, disabled/unhealthy provider, fallback, cost/quality priority
Tool gate	registered vs dispatchable, missing tool, unauthorized write tool
Context	minimum authorized context, stale version, cross-project/tenant block
Execution	timeout, invalid result, schema validation, exact artifact refs
QA independence	creator cannot validate self; changed build/version re-QA
Retry	safe retry, bounded budget, backoff, correlation preservation
Fallback	equivalent capability, policy/permission preserved
Side effects	message/build/Figma/invoice duplication protection
Finance gate	proof/match does not become Admin verification; Phase 5 blocked until verified event
Escalation	no-route, policy conflict, repeated failure, ambiguous side effect
Admin trace	routing/execution/failure/cost/audit visible
Regression	Phase 1-3 routing foundations remain functional

32. Mandatory Detailed Test Cases

ID	Scenario	Expected result
OR4-T001	PM communication task	Routes to PM-capable specialist/model
OR4-T002	Initial full UI task with valid baseline	Routes to UI Designer
OR4-T003	UI Designer task with invalid activation	Blocked/no design route
OR4-T004	Ambiguous business rule	No Designer guess; clarification route
OR4-T005	Design QA task	Routes to independent QA identity
OR4-T006	Designer selected as own validator	Rejected
OR4-T007	Admin UI approval missing but PM share requested	Blocked
OR4-T008	Prototype build before client UI approval	Blocked
OR4-T009	Prototype build with exact locked UI	Routes to Prototype Agent
OR4-T010	Prototype QA task	Routes to independent validation agent
OR4-T011	Prototype Agent selected as own validator	Rejected
OR4-T012	Client prototype share before Admin approval	Blocked
OR4-T013	Phase4Completed valid	Routes M2 Finance task
OR4-T014	Finance attempts final Admin verification	Authority validation rejects
OR4-T015	Task 3 requested on PaymentSubmitted	Blocked
OR4-T016	Task 3 requested on M2PaymentVerified	Allowed route
OR4-T017	Disabled specialist	Filtered; safe fallback/escalation
OR4-T018	Provider unhealthy	Alternative eligible provider selected
OR4-T019	No provider supports required tool/schema	Escalate
OR4-T020	Model cheapest but below quality policy	Not selected
OR4-T021	Unauthorized tool requested	Blocked before tool call
OR4-T022	Tool registered but non-dispatchable	Blocked
OR4-T023	Cross-tenant context ref	Denied
OR4-T024	Stale UI version	Reject/reconcile exact version

ID	Scenario	Expected result
OR4-T025	Invalid specialist output schema	Reject; retry/fallback if safe
OR4-T026	Provider says success but artifact missing	Treat incomplete
OR4-T027	Timeout on safe read/generation	Retry within budget
OR4-T028	Uncertain client-message side effect	Reconcile before retry
OR4-T029	Duplicate PrototypeBuildRequested event	No duplicate build side effect
OR4-T030	Duplicate Phase4Completed event	No duplicate M2 invoice path
OR4-T031	Fallback provider used	Same specialist/domain authority and correlation preserved
OR4-T032	Retry budget exhausted	Durable escalation
OR4-T033	Conflicting authoritative states	Stop and escalate
OR4-T034	Provider output tries to grant permission	Ignored/rejected
OR4-T035	Secret appears in model/tool log	Security FAIL
OR4-T036	Routing trace requested	Complete selection/execution/fallback history available
OR4-T037	Budget threshold reached	Apply configured lower-cost/escalation policy
OR4-T038	Admin disables provider route	New tasks no longer route there
OR4-T039	Changed prototype build carries old QA PASS	Rejected; revalidation required
OR4-T040	Same policy/input repeated	Deterministic/consistent eligible routing within configured tie-breaking rules

33. Mandatory Phase 4 Orchestrator E2E Scenario

- ☐ Receive valid Phase3Completed / Task2Started workflow task.
- ☐ Route PM Task 2 start communication; preserve delivery side-effect idempotency.
- ☐ Route initial full UI task to UI Designer with exact Phase 3 baseline.
- ☐ Designer returns exact UI version metadata.
- ☐ Route UI to independent Design QA.
- ☐ QA returns CHANGES_REQUIRED; route exact defects back to Designer.
- ☐ Route revised exact UI back to QA; validate QA PASS.
- ☐ Workflow/Admin approves; route PM exact-version client share.
- ☐ Client approves UI; route Prototype build from exact locked UI.
- ☐ Prototype Agent returns build metadata; route exact build to Prototype Validation QA.
- ☐ QA finds broken route; route defects to Prototype Agent.
- ☐ Route revised build back to QA; validate QA PASS.
- ☐ Workflow/Admin approves; route PM exact-build client share.
- ☐ Client final approval leads workflow to Phase4Completed.
- ☐ Route Finance M2 task.
- ☐ Finance produces invoice/payment workflow; client submits proof.
- ☐ Attempt Task 3 before Admin verification - Orchestrator blocks.
- ☐ Admin verification produces authoritative M2PaymentVerified.
- ☐ Route PM Task 3 start / Phase 5 eligible task.
- ☐ Replay key events and prove no duplicate message/build/M2 side effects.
- ☐ Simulate one provider outage and prove approved provider fallback with full trace.
- ☐ All task IDs, correlation IDs, agents, models/providers, versions, attempts and results remain visible in Admin trace.

34. Definition of Done - Phase 4 Orchestrator / Router

- ☐ Phase 4 task taxonomy and capability registry are implemented.
- ☐ Correct specialist is selected for PM/Designer/QA/Prototype/Finance work.
- ☐ Conditional Designer routing works.
- ☐ Capability-driven model/provider selection follows configured policy.
- ☐ Exact phase/UI/build/payment guards run before dispatch.
- ☐ Execution envelope contains authorized minimum context and exact version refs.
- ☐ Tool permission/dispatchability gate is enforced.
- ☐ Structured outputs and authority provenance are validated.
- ☐ Creator cannot self-validate through routing.
- ☐ Retry/fallback are bounded, safe and traceable.
- ☐ Side effects are protected by idempotency/reconciliation.
- ☐ Admin/client/payment gates cannot be bypassed by AI output.
- ☐ Finance final verification remains human/Admin authority.
- ☐ Cross-tenant/project context and secrets are protected.
- ☐ No-route/policy/repeated failures create durable escalation.
- ☐ Admin can inspect route/execution/failure/fallback/usage traces.
- ☐ Phase 4 E2E and negative tests pass.
- ☐ No unresolved Phase 4 Orchestrator P0/P1 defects remain.

35. Master Orchestrator Checklist

A. Intake

- ☐ task/correlation
- ☐ tenant/project
- ☐ task type
- ☐ intent
- ☐ schema

- ☐ versions
- ☐ policy
- ☐ classification
- ☐ idempotency

B. Capability

- ☐ agent registry
- ☐ task support
- ☐ schemas
- ☐ tools
- ☐ permissions
- ☐ autonomy
- ☐ health
- ☐ constraints

C. Routing

- ☐ eligibility filters
- ☐ state guards
- ☐ provider candidates
- ☐ policy
- ☐ quality
- ☐ cost
- ☐ latency
- ☐ fallback

D. Envelope

- ☐ context refs
- ☐ artifact versions
- ☐ criteria
- ☐ tools
- ☐ policy refs
- ☐ timeout
- ☐ retry budget
- ☐ security

E. Context

- ☐ minimum necessary
- ☐ canonical records
- ☐ provenance
- ☐ stale detection
- ☐ tenant isolation

F. Tools

- ☐ registered
- ☐ dispatchable
- ☐ authorized
- ☐ argument validation
- ☐ external response validation

G. Execution

- ☐ persist decision
- ☐ dispatch
- ☐ timeout
- ☐ status

- ☐ tool trace
- ☐ result ingestion
- ☐ schema validation

H. QA Independence

- ☐ Designer not own QA
- ☐ Prototype Agent not own QA
- ☐ changed artifacts re-QA

I. Reliability

- ☐ failure classification
- ☐ retry
- ☐ backoff
- ☐ fallback
- ☐ idempotency
- ☐ uncertain side-effect reconciliation

J. Finance Gate

- ☐ Phase4Completed only
- ☐ Finance route
- ☐ Admin verification provenance
- ☐ Task3 after M2PaymentVerified

K. Security

- ☐ RBAC
- ☐ tenant/project scope
- ☐ data classification
- ☐ secret isolation
- ☐ audit

L. Observability

- ☐ route trace
- ☐ execution trace
- ☐ attempts
- ☐ failures
- ☐ fallbacks
- ☐ cost/usage
- ☐ escalations

M. Admin Controls

- ☐ agent status
- ☐ provider routing
- ☐ fallback
- ☐ budgets
- ☐ timeouts
- ☐ tool permissions
- ☐ route enable/disable

N. Testing

- ☐ unit
- ☐ integration
- ☐ event/job
- ☐ tool
- ☐ security
- ☐ negative

- ☐ fallback
- ☐ E2E
- ☐ regression

36. Final Locked Phase 4 Operating Contract

UNDERSTAND TASK → VALIDATE PHASE/ARTIFACT GATES → MATCH SPECIALIST CAPABILITY → FILTER BY PERMISSION / DATA / TOOL → SELECT ELIGIBLE MODEL/PROVIDER → BUILD EXECUTION ENVELOPE → DISPATCH → VALIDATE RESULT + AUTHORITY PROVENANCE → RETRY/FALLBACK SAFELY → RETURN TRACEABLE

Always: route the right work to the right specialist through an authorized, policy-compliant, traceable execution path.
Never: become the specialist, invent business facts, bypass Admin/client/payment gates, leak context/secrets, self-authorize actions, or blindly retry uncertain side effects.